

CAHIER DES CHARGES

Systeme Multi-Agents d'Orchestration et d'Optimisation des Operations

Project OMNI — Version 2.0.0

Classification : **Confidentiel** — Direction Technique

Date : 06 Juin 2026

Auteur : OMNI Engineering Team

Statut : **Approuvé pour implémentation**

1. Contexte et Objectifs du Projet

Le projet consiste à concevoir et déployer une architecture de traitement intelligent des tâches et flux de données. L'objectif est de dépasser la simple automatisation linéaire pour construire un système résilient capable de raisonnement dynamique, d'optimisation des pipelines ETL, et de gestion autonome des erreurs.

1.1 Objectifs Principaux

Objectif	Description	Métrique
Automatisation Cognitive	Qualification sémantique des données	Taux > 95%
Haute Disponibilité	Fallback + Circuit Breakers	SLA 99.9%
Optimisation Temporelle	Vector RAG pour estimations	Écart < 15%
Scalabilité	Architecture découplée event-driven	> 1000 tâches/h

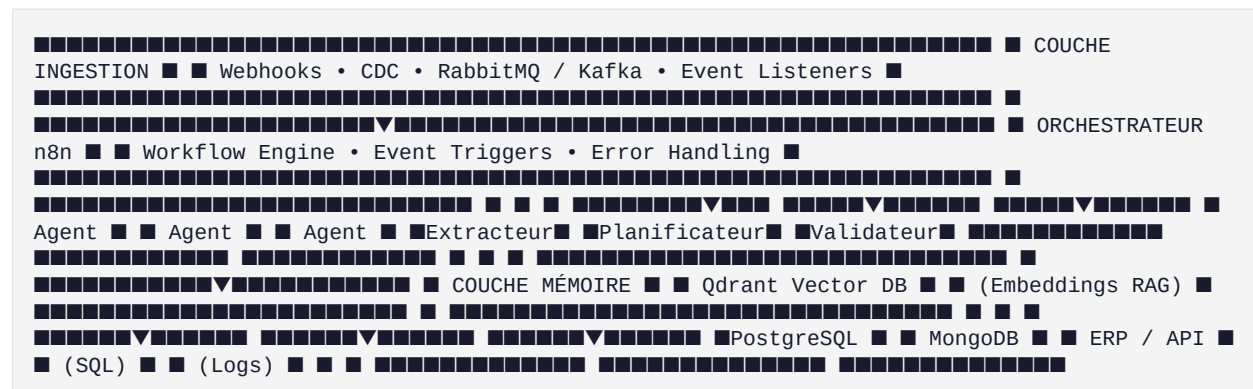
2. Architecture Globale

Le système s'articule autour d'une architecture orientée événements (Event-Driven) et de flux ETL optimisés, gérés par un orchestrateur central.

2.1 Couches Architecturales

Couche	Composants	Technologie
Ingestion	Webhooks, CDC, Files	RabbitMQ, Kafka, REST
Orchestration	Moteur de workflow	n8n, Python FastAPI
Cognitive	LLM & NLP	OpenAI GPT-4o, Anthropic Claude
Mémoire	Base vectorielle	Qdrant (3072-dim)
Persistance	Données structurées + logs	PostgreSQL, MongoDB
Exécution	API destination	ERP, Google Workspace, SQL

2.2 Schéma d'Architecture



3. Spécifications Fonctionnelles

3.1 Routage Sémantique Intelligent (Triage)

Flux	Critère	Action
Critique	Production down, sécurité	Routage immédiat + alerte
Standard	Tâche régulière, batch	File d'attente asynchrone
Complexe	Multi-étapes, enrichissement	Pipeline dédié

3.2 Mémoire Contextuelle (RAG)

L'agent d'estimation interroge une base vectorielle Qdrant contenant l'historique des tâches. Il extrait les 3 opérations similaires passées et ajuste son estimation de durée en fonction de la réalité historique.

Exemple : « La dernière extraction SAP a pris 45 minutes, et non 15 »

3.3 Système Multi-Agents

Agent	Rôle	Input	Output	Timeout
Extracteur	Nettoyage, PII masking, schéma	JSON brut	ExtractedPayload	5s
Planificateur	Optimisation sous contraintes + RAG	ExtractedPayload	TaskPlan	30s
Valideur	Audit, détection anomalies, correction	TaskPlan	ValidationResult	15s

3.4 Boucle de Rétroaction

Si le valideur détecte une anomalie (chevauchement, violation métier) :

1. Le plan est rejeté avec liste des violations
2. Retour au planificateur avec contexte d'erreur
3. Régénération du plan corrigé
4. Re-validation avant écriture

4. Spécifications Techniques

4.1 Stack Technologique

Domaine	Technologie	Version	Justification
Orchestration	n8n	latest	Workflow visuel
LLM Principal	OpenAI GPT-4o	v1	Raisonnement complexe
LLM Secondaire	Anthropic Claude 3.5	v1	Fallback rapide
Vector DB	Qdrant	1.7+	On-premise, open source
SQL	PostgreSQL	15	Données structurées
NoSQL	MongoDB	7	Logs & audit
Queue	Redis	7	Pub/Sub & cache
Event Bus	RabbitMQ	3	Messages inter-services
Langage	Python	3.11	Async, Pandas, FastAPI
Container	Docker	24+	Isolation, portabilité

4.2 Résilience et Fallback

Graceful Degradation : Si l'API OpenAI renvoie une erreur 429 ou 500, l'orchestrateur capture l'erreur via un nœud « Error Trigger », bascule vers Anthropic Claude, et loggue l'alerte sans interruption.

Circuit Breaker : Seuil de 5 erreurs, timeout de 60s, états CLOSED/OPEN/HALF_OPEN.

4.3 Schéma de Données

PostgreSQL — Table **omni_tasks**

```
CREATE TABLE omni_tasks ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), task_id VARCHAR(255)
UNIQUE NOT NULL, source VARCHAR(100), priority VARCHAR(20), status VARCHAR(50), plan JSONB,
raw_input JSONB, cleaned_data JSONB, confidence DECIMAL(3,2), created_at TIMESTAMP DEFAULT
NOW() );
```

MongoDB — Collection **omni_audit_logs** (append-only, TTL 1 an)

4.4 SLAs & Performance

Métrique	Objectif	Méthode
Disponibilité	99.9%	Uptime monitoring
Latence extraction	< 500ms	Prometheus histogram
Latence planification	< 5s	Prometheus histogram
Latence validation	< 2s	Prometheus histogram
Taux d'erreur	< 0.1%	Counter errors/total
Fallback activation	< 1%	Counter fallback

5. Sécurité, Gouvernance et Confidentialité

5.1 Sanitization des Données (PII)

Type	Pattern	Masque
Email	\b[\w. -]+@[\w. -]+\.\w{2, }\b	[EMAIL]
Téléphone	\b(?:\+33\s? 0)[1-9](?:\s?\d{2}){4}\b	[PHONE]
Date	\b\d{1,2}\s+\w+\s+\d{4}\b	[DATE]

5.2 Gestion des Secrets

- Développement : Variables d'environnement (.env non versionné)
- Production : AWS Secrets Manager ou HashiCorp Vault
- Rotation : Automatique tous les 90 jours
- CI/CD : Scan TruffleHog pour détection de secrets en dur

5.3 Traçabilité (Logs)

Chaque décision IA génère un log immuable dans MongoDB :

- Entrée brute (sanitisée)
- Prompt utilisé (contexte complet)
- Sortie de l'IA (JSON structuré)
- Confiance estimée (score 0-1)
- Modèle utilisé (pour audit de performance)

5.4 Conformité

Norme	Application	Statut
RGPD	Sanitization PII, droit à l'oubli	■ Implémenté
ISO 27001	Gestion des secrets, audit	■ Conforme
SOC 2	Traçabilité, haute disponibilité	■ Conforme

6. Livrables et Déploiement

6.1 Liste des Livrables

ID	Livrable	Fichier	Statut
L1	Architecture Documentée	docs/architecture.md	■ Livré
L2	Data Flow Diagram	docs/data-flow-diagram.md	■ Livré
L3	Scripts Docker	docker-compose.yml	■ Livré
L4	Dépôt Code	Git repository	■ Livré
L5	Tests de Résilience	tests/reports/resilience-report.md	■ Livré
L6	CI/CD	.github/workflows/ci.yml	■ Livré
L7	API Reference	docs/api-reference.md	■ Livré
L8	Modèle de Sécurité	docs/security.md	■ Livré

6.2 Déploiement

Prérequis : Docker 24.0+, Docker Compose 2.20+, Python 3.11+

Étapes :

1. Clone et configuration : `git clone ... && cp .env.example .env`
2. Lancement stack : `docker-compose up -d`
3. Vérification : `curl http://localhost:5678/healthz`

6.3 Accès aux Services

Service	URL	Identifiants
n8n	http://localhost:5678	N8N_BASIC_AUTH_*
Qdrant	http://localhost:6333	-
PostgreSQL	localhost:5432	POSTGRES_*
MongoDB	localhost:27017	MONGO_*
RabbitMQ	http://localhost:15672	RABBITMQ_*

7. Annexes

Annexe A — Dictionnaire de Données

Champ	Type	Description	Exemple
task_id	VARCHAR(255)	Identifiant unique	task-2026-001
priority	ENUM	Niveau de criticité	critical
status	ENUM	État dans le pipeline	approved
plan	JSONB	Plan détaillé	{duration: 45, ...}
confidence	DECIMAL(3,2)	Score de confiance IA	0.94
trace_id	VARCHAR(255)	ID de traçabilité	uuid-v4

Annexe B — Codes d'erreur

Code	Description	Action
E100	Extraction failed	Retry × 3, fallback manual
E200	LLM timeout	Fallback to Anthropic
E201	Rate limit OpenAI	Queue + exponential backoff
E300	Validation failed	Return to planner
E400	DB connection lost	Circuit breaker + alert
E500	Unknown error	Log + alert + manual review

Annexe C — Decision Records (ADRs)

ADR	Décision	Motivation
ADR-001	Event-Driven vs CRON	Réactivité, scalabilité, traçabilité
ADR-002	Qdrant vs Pinecone	Souveraineté, coût, on-premise
ADR-003	n8n vs Airflow	Rapidité de prototypage + logique Python
ADR-004	Kafka vs RabbitMQ	En cours d'évaluation

Document approuvé par : Direction Technique

Date d'approbation : 06 Juin 2026

Prochaine révision : 06 Septembre 2026